

6. Une comparaison est un nombre

« `x == 7` » n'est pas une question posée dans le vide : c'est un calcul qui donne 1 ou 0.

1. Ce qu'il faut comprendre

Les six comparaisons : `==` (égal), `!=` (différent), `<` `>` (plus petit, plus grand), `<=` `>=` (plus petit **ou égal**, plus grand **ou égal**).

Chacune **se calcule** comme une addition, et donne un nombre : **1 si c'est vrai, 0 si c'est faux**. On peut donc l'afficher avec `nombre()`, la ranger dans une variable, l'additionner. `if` ne fait rien d'autre que regarder si ce nombre vaut 0.

Le piège : `=` range, `==` compare. `if (x = 7)` range 7 dans x, et comme 7 n'est pas zéro, la condition est toujours vraie. C'est l'erreur de frappe la plus sournoise du C++ : le programme compile, et fait autre chose.

`x < 10` et `x <= 9` disent la même chose sur des entiers. Choisis celle qui se lit le mieux : « tant que x est plus petit que la largeur » s'écrit `x < LARGEUR`.

À toi : change la valeur de `x` en 10 et prédis chaque ligne avant de compiler.

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **1057 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

```
démarrage          49 o ← la console, à l'allumage — la mise en route de la console
main()             266 o ← démarrage — ta fonction
VBlank              8 o ← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage       28 o ← image() — attend l'image suivante
AttendreVBlank      16 o ← démarrage — attend que l'écran soit libre
AttendreAcces       12 o ← EcrireTexte, EcrireNombre — attend de pouvoir écrire une case
EcrireTexte         10 o ← texte() — écrit un texte, lettre par lettre
EcrireNombre        33 o ← nombre() — écrit un nombre en chiffres
CopierTuiles        85 o ← démarrage — copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage — vide les deux cartes de l'écran
Tuiles              432 o ← CopierTuiles — 27 lettres sur 43
"X VAUT 7"          8 o ← texte() — un texte gravé
"X EGAL 7"           8 o ← texte() — un texte gravé
"X DIFFERENT DE 7"  16 o ← texte() — un texte gravé
"X PLUS PETIT 10"   15 o ← texte() — un texte gravé
"X PLUS GRAND 10"   15 o ← texte() — un texte gravé
"X AU PLUS 7"       11 o ← texte() — un texte gravé
"X AU MOINS 8"      12 o ← texte() — un texte gravé
"VRAIES ADDITION..." 19 o ← texte() — un texte gravé
```

Lettres gravées : ADEFGILMNOPRSTUVX0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, LireManette, DonneesTransfert, Notes

Fonctions du langage appelées :

`nombre() × 7` 124 o sur place, dans `main()` + `EcrireNombre` (33 o)

```
texte()  x 8    88 o sur place, dans main() + EcrireTexte (10 o)
image()  x 1    3 o sur place, dans main() + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// CE PROGRAMME : pose des questions sur x (x vaut 7) et affiche les réponses.
// Une comparaison donne un NOMBRE : 1 si c'est vrai, 0 si c'est faux.
//
// == égal à (DEUX signes « = » : un seul « = » veut dire « ranger dans »)
// != différent de
// < plus petit que > plus grand que
// <= plus petit ou égal >= plus grand ou égal

#include <texte> // écrit un texte à l'écran
#include <nombre> // écrit un nombre en chiffres
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() { // Le jeu commence ici.
    uint8_t x = 7; // On range 7 dans x.

    texte(1, 1, "X VAUT 7"); // Rappel à l'écran.

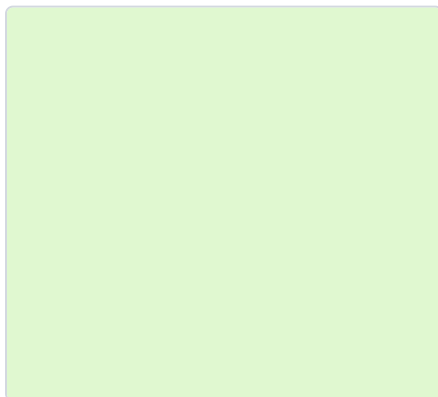
    // nombre(colonne, ligne, valeur, 1) : le dernier 1 veut dire « écris 1 seul chiffre ».
    texte(1, 4, "X EGAL 7"); nombre(18, 4, x == 7, 1); // 7 égal à 7 ? oui : 1
    texte(1, 5, "X DIFFERENT DE 7"); nombre(18, 5, x != 7, 1); // 7 différent de 7 ? non : 0
    texte(1, 6, "X PLUS PETIT 10"); nombre(18, 6, x < 10, 1); // 7 plus petit que 10 ? 1
    texte(1, 7, "X PLUS GRAND 10"); nombre(18, 7, x > 10, 1); // 7 plus grand que 10 ? 0
    texte(1, 8, "X AU PLUS 7"); nombre(18, 8, x <= 7, 1); // 7 au plus 7 ? oui : 1
    texte(1, 9, "X AU MOINS 8"); nombre(18, 9, x >= 8, 1); // 7 au moins 8 ? non : 0

    // Puisqu'une réponse est un nombre, on peut les additionner :
    // (x == 7) vaut 1, (x < 10) vaut 1, (x <= 7) vaut 1 → 1 + 1 + 1 = 3.
    uint8_t vraies = (x == 7) + (x < 10) + (x <= 7);
    texte(1, 12, "VRAIES ADDITIONNEES");
    nombre(18, 13, vraies, 1); // affiche 3

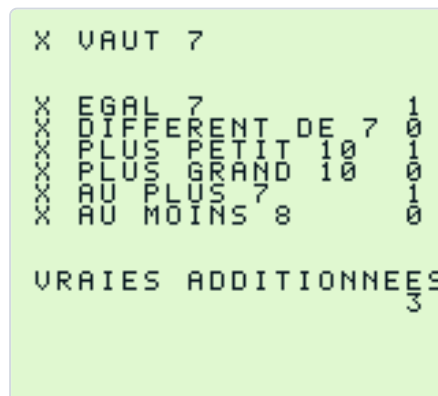
    while (true) { // Répète pour toujours :
        image(); // attend l'image suivante.
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée.



la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

Une colonne de 1 et de 0 : 1, 0, 1, 0, 1, 0 — et 3 quand on additionne trois comparaisons vraies.

5. Ce que la console répond

Ces contrôles sont ceux que `verification/verifier-cours.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ chaque comparaison a donné 1 ou 0 (101010)

✓ trois comparaisons vraies s'additionnent en 3

← [5. if et else : deux chemins](#)

[sommaire](#)

[7. Et, ou, non : && || !](#) →